论文

SYsU-lang：步步为营，构建编译实践全局观

吴坎，王永康，刘皓铧，张献伟*

中山大学计算机学院, 广东省广州市广州大学城外环东路 132 号 510006
* 通信作者. E-mail: zhangxw79@mail.sysu.edu.cn

摘要 针对目前编译实验教学过程中内容安排不够合理、与实际应用契合不够紧密的问题，我们提出了基于 Clang/LLVM 面向业界实际的实验教学模式。新的实验模式要求面向精简 C 语言以尽量少的代码实现一款相对完整的编译器，在实现过程中可随时与 Clang/LLVM 进行交互和验证，并可在自动评测机上进行即时性能测试反馈与排名。相比同类课程，提出的实验设计更好的平衡了基础性、实践性和综合性，使得学生可以通过实际编译框架，更好的理解编译机制，掌握利用编译解决实际问题的能力，构建起编译实践的全局观。

关键词 编译原理；系统实践；实验教学；Clang；LLVM

1 引言

编译器是计算机系统软件的一个重要分支，对构建从程序编写到硬件执行的完整系统认知起着举足轻重的作用。作为从理论到实践的典范，编译成为计算机专业的核心基础课程，同时也是计算机专业中最具挑战性的课程之一。并重理论与实践，编译课程要求学生既理解和掌握基本原理、编译机制和算法，又需要学生掌握具体设计、实现一个基本编译器的初步能力。然而，由于编译理论抽象，系统实现复杂，导致学生普遍认为编译课程枯燥、繁琐、无用，缺乏学习热情，失去积极性和主动性。

本科编译教学内容繁多，涉及诸多理论体系，覆盖词法分析、语法分析、语义分析、中间代码生成、代码优化和目标代码生成等数个阶段，包含自动机、文法语言、控制流图等经典模型。如果缺乏相应的实践练习，理论学习将极易形式化、空洞化。编译实验作为将理论知识转化为实践经验的良好载体，对支撑理论教学、提高实践能力具有重要意义。因此，编译实验得到了普遍的重视，被众多国内外高校列为单独内容或与编译原理并列的单独课程（例如，中山大学计算机学院单独开设 36 学时的编译器构造实验课程）。良好的编译器构造实验极为关键，可以促使学生通过对各编译阶段的实现，提高动手实践能力，同时加深对编译基础性理论的理解。

然而，目前的编译实验存在囿于理论、系统不足的问题，例如围绕表达式计算（或算术计算器）展开的实验导致学生无法建立对编译器和编译流程的基本性认知。对此问题，已有诸多学校编译实验要求学生针对具体编程语言自主构建相对完整的编译系统。然而，这也存在一定的问题：首先，从

零开始分别动手实现并组装成完整编译器极为繁琐与困难，导致工作量大而难以被学生在学期内完成；同时，面向小型虚拟语言（例如，MiniJava）的自主编译开发，造成学生很难与实际主流编译器（例如，GCC、LLVM）和实用高级编程语言（例如，C、C++）建立关联，实验践积累无法有效用于实际。此外，作为教育部“101”计划 [1] 所列的 12 门核心课程之一，编译课程对实验教学提出了更高的要求，需要面向实践和实用以夯实学生基础，加强紧缺人才培养 [2]，实现教育链、产业链和创新链深度融合 [1]。

Clang/LLVM 是最为流行的编译构架之一，衍生了大量的定制化编译工具，广泛应用于产业界和学术研究，形成了具有实际标准意义的生态。因此，Clang/LLVM 提供了面向实际编译教学的理想平台，可有力促进理论学习和动手实践的结合。我们将探索以 Clang/LLVM 为依托串联关键编译阶段，呈现清晰的知识体系和学习路径，并进一步融合理论教学和自主探究式学习，充分发挥学生的主体作用，提升编译教学效果。对比现行课程安排，我们设计的编译实验具有以下特点和优势：

- 面向实际需求和开源平台: 实验基于 LLVM 开源编译器分阶段、层次化展开，学生可通过实验熟悉和掌握编译技术和工具，并利用开源社区自发学习，能够更好的将所学应用于解决工作和科研中的实际问题。
- 突出重点，构建全局观: 各实验覆盖了编译的主要模块，且考虑到工作量和学习效果，更聚焦于编译前端和 IR（Intermediate Representation，中间代码表示），而淡化更偏硬件的后端，以帮助学生迅速对编译整体流程和核心模块建立全局认知。同时，为方便上手，我们对各实验提供了精简模版代码，且控制总体代码 3000 行左右，并做到各实验高度解耦化，可随时与标准 Clang/LLVM 进行交互和验证。
- 强调基础，鼓励探索: 设计的实验在强调对编译基础技术掌握的同时，在各阶段均提供了扩展性实验建议和指引，供学有余力的学生进行创新探索；此外，实验中使用的教学语言 SYsU 是 C 语言的精简子集，因此学生可将设计的编译器拓展于其他系统课程，包括组成原理、操作系统等，从而可以建立更为完善的计算机系统观。

2 现状分析

2.1 业界发展与需求

LLVM 是最为流行的开源编译器框架，支持多种语言和底层硬件。LLVM 可被视为一个模块化的、可重用的编译器和工具链集合，包含了完备的开发库方便进行二次开发扩展支持新语言和架构，以及定制实现特定优化功能。Clang 是一个 C++ 编写、基于 LLVM、发布于 LLVM BSD 许可证下的 C/C++/Objective-C/Objective-C++ 的编译器，可被视为 LLVM 编译工具集的前端部分。相比于传统 GCC 编译器，Clang/LLVM 在使用上具有编译速度更快、占用内存更小、诊断信息可读性更强的优势 [3]。而在代码结构上，Clang/LLVM 有更完善的基于库的模块化设计，代码更清晰简单，易于理解和扩展。

近些年，随着计算机系统和应用的飞速发展，基于 Clang/LLVM 的编译技术得到了广泛的应用，推进编译器研发进入黄金时代 [4]。在对 X86 和 ARM 等主流架构 CPU 完善支持以外，LLVM 在以 GPU 为代表的加速器上也得到了普遍的采用，包括 NVIDIA 和 AMD 基于 LLVM 二次开发的

表 1　编译课程实验对比（注：O 代表可选或部分支持）

课程	面向语言	前端	中端（IR）	后端	自顶向下	与 LLVM 对接	自动评测
Stanford CS143 [21]	Cool	Y		Y	Y		Y
Cornell CS6120 [22]	Bril	Y	Y		Y	Y	
国内课程 [23∼25]	C-like	Y	O	Y			Y
** 理想实验 **	C-like	Y	Y	O	Y	Y	Y

`nvcc` [5] 和 `hcc` [6]，以及 Google TPU Compiler [7]。得益于性能优势和完整生态，LLVM 也将被 Intel 和 IBM 采用以更好的支持下一代计算 [8, 9]。与此同时，随着人工智能、区块链、量子计算等新型应用的迅速发展，编译技术，特别是 LLVM 框架本身，也被进一步研究和使用以更好的适配新场景、新需求，包括编译框架多级中间表示 MLIR [10]，端到端深度学习编译器 TVM [11]，区块链合约验证 [12,13], 量子计算编程 [14,15]，等。国内系统和处理器厂商同样在基于 LLVM 编译技术进行国产化的支持和完善，例如华为毕昇编译器 [16] 和百度飞桨深度学习框架 [17]。

如上所述，产业界和学术界大量编译相关的工作基于 Clang/LLVM 实施，因此从实践和实用的角度讲，编译教学有必要针对性开展实验设计，为学生后续工作和深造奠定基础。以 Clang/LLVM 组织实验教学，不仅可以让学生更深刻的理解现代编译器的整体架构和流程，还能锻炼学生面向开源技术的自主式、探索式学习，培养改进编译系统软件的基本能力，开拓独立承担实践任务的能力。

2.2　国内外相关课程

立足于实践和实用的同时有效促进理论教学，编译实验需合理设置，平衡覆盖面和工作量，使得学生可以在有限的时间内尽量高效的完成实验流程。为更好的启发课程设计，我们综合对比分析国内外实验课程，包括面向的编程语言、覆盖的编译阶段、是否基于 LLVM 进行开发，以及是否提供自动评测机制等。对比结果见表-1。

Stanford CS143: 斯坦福（Stanford）大学 CS143 课程是面向虚拟 Cool 语言，由词法分析、语法分析、语义分析和 MIPS 目标代码生成四个基本实验，和可选的机器代码优化实验或熟悉 Cool 语言的上手实验（MOOC 版）构成，不依赖 Clang/LLVM，各部分实现需要学生在提供的代码框架中完成。以上实验设置存在偏离实际编程语言和工具链的问题，需要额外的上手熟悉过程，且过于关注机器代码生成，而忽视了更为精髓的中间代码生成与优化。

Cornell CS6120: 康奈尔（Cornell）大学的高级编译课程则更注重编译环节，实验同样划分为 4 个部分，分别是：小型语言 Bril 生态扩展、编译优化（语法树层面）、编写 LLVM Pass，以及自选主题的开放项目。虽有涉及部分编译环节且引导学生熟悉 LLVM Pass，但实验总体缺乏完整性，各部分耦合不够紧密，未能较好的做到分阶段覆盖主要环节，因此不能有效帮助学生建立编译的完整认知。与此同时，LLVM Pass 编写是基于旧的 Pass 机制 [20]，且在评测集上需要学生自主构建并编写评测脚本，明显增加了工作量和繁琐度。

国内课程（清华大学、北京大学、北京航空航天大学等）：过去几年，随着国内对计算机系统教育的愈发重视，代表性高校的编译课程都围绕实践和实用方面进行了实验教学改革，例如面向类 C 编程语言、基于 Clang/LLVM。通过对几所典型高校的实验教学调研，我们发现：1）教学语言方面，普遍基于全国大学生计算机系统能力大赛——编译系统设计赛中使用的类 C 语言 SysY [26]，以赛促

改、以赛促学。然而，SysY 语言精简了预处理（`#define`、`#include` 等）过程，在实现编译器自举或扩展编译实现支持其他应用上缺乏表现力，存在较大的局限性；2）实验开展上，清北和北航的课程要求学生继续向已有的实现里添加语法来推进实验，但学生因初期不具备完整知识体系，其代码随语法逐渐复杂化可能需要不断进行重构，如图 7；3）Clang/LLVM 对接上，清北都提供了较为复杂的实验框架或自定义的 IR 表示，而非直接基于 LLVM 进行开发，这也阻碍学生对编译核心流程认知的建立；4）编译最终输出方面，清北要求完成 RISC-V 汇编代码的生成，其他则不基于 LLVM 库最终输出 LLVM IR，这样会导致任务相对繁重，极大增加了工作量；5）测评机制方面，普遍包含自动评测脚本，但评测机部分通常不开源，难以在课程中直接利用并改进。

3 设计理念与目标

结合前述分析，可以发现基于 Clang/LLVM 进行实验教学能够更好地突出编译的实践性和实用性，便于学生利用开源技术和文档进行自主学习，且利于将所学拓展运用于实际。然而，考虑到教学效果，基于 Clang/LLVM 的实验开展需要精细化考虑，平衡多方面需求。首先，实验设计应该兼顾动手实践和全局把握，争取让学生用尽量简洁清晰的代码完成关键编译流程的实现，对完整编译机制和技术建立全局认识。其次，各分部实验应该尽量模块化和解耦化，做到与 LLVM 可以灵活交互和分离，方便与标准版本进行对比。此外，实验环境应该尽量友好和完善，便于上手和及时得到反馈，因此完整精简的开发环境和及时的自动集成测试可以作为重点考虑。在国内外高校相关课程基础之上，我们将结合中山大学计算机学院教学实际，发挥后发优势，围绕以下几点设计课程实验：

基于理论教学，互补理论教学：编译理论教学覆盖从词法分析到目标代码生成的完整阶段，但绝大部分学时被投入到原理性知识学习中，包括自动机转换和语法分析机制等，而大幅淡化了 IR 及之后更偏实践阶段。因而，词法分析和语法分析等核心编译环节必须落实于实验教学，形成对理论学习的加强和互补。与此同时，理论教学淡化的 IR 及优化在实际工作和学习中应用最为广泛，因此需要在实验中进行侧重。综上，我们的实验设计在覆盖经典词法语法等阶段的同时，将更于聚焦 IR 和优化，突出实用性。

过程模块化，全局完整化：与现有多数课程实验相似，我们将整体编译实验切分为几个部分，从编译的前端向后端逐次推进。然而，考虑到各实验在输入输出上互为依赖（例如，语法分析以词法分析结果作为输入，输出 AST（Abstract Syntax Tree，抽象语法树）供 IR 生成阶段使用），容易导致学生在前期某个实验环节完成不够理想的情况下严重影响后续实验的开展。针对此问题，我们将各阶段模块化兼解耦化，不必后向依赖，可随时与 Clang/LLVM 交互，即输入可从 Clang/LLVM 相应阶段获取。以 LLVM 为蓝本，从少量模版代码开始逐步完成功能开发，最终实现一个功能较为完整的编译器；同时，实现过程中可参阅 LLVM 官方文档及代码，建立编译器构建的全局观。

完善开发环境，降低完成门槛：我们认为，实验设计与难度不是证明教师和助教能力的舞台，而应该引导学生进行难度适宜的开发实践。考虑到课时的限制，编译实验在保证上述教学效果的同时整体工作量不宜过大，否则极易导致相当比例的学生无法正常跟上教学进度。为此，我们的实验设计借助 LLVM 库精简开发流程，定位于三千行左右实现一款相对完整的编译器，将目标代码生成设为可选以更凸显关键编译环节。同时，为方便学生上手，每个实验我们都提供了一定的模板代码以及完整的编译脚本，使得学生可以通过动手尝试快速进入实验状态。需要强调的是，所提供的模板

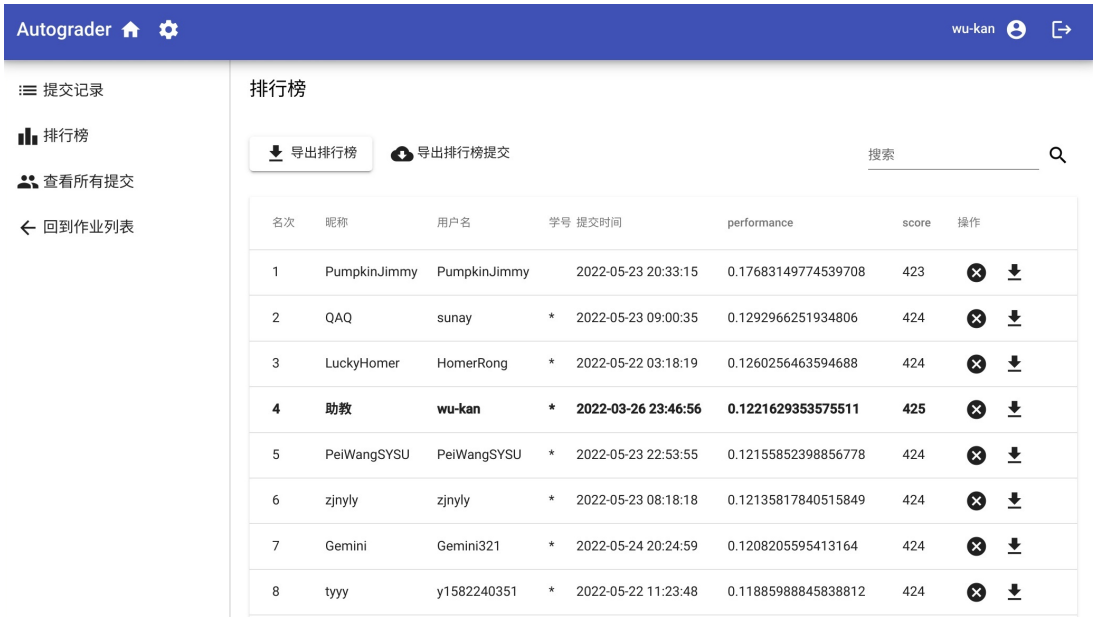

图 1　在线评测与性能排名

代码通常不超过 100 行，从而方便学生对后续代码实现建立更为清晰的认知。此外，我们提供了完整的性能测评机制和排名（见图-1），以及详尽的测试用例，便于学生第一时间得到反馈，以用例促进实验开展。

重在基础、鼓励探索：鉴于本科核心课程教育的基础性定位，本实验设计亦定位于编译经典知识的掌握，和通用编译技术的熟悉，而不刻意追求章节-2.1 所述的 TVM 等前沿性内容。但考虑到部分学有余力学生的实际需要，我们充分鼓励在每个阶段性实验进行自由拓展和探索，并提供若干挑战问题和扩展建议，例如脱离 `bison` 自主实现文法解析、基于 AST 实现代码重构工具。同时，代码优化实验也不施加过多约束，鼓励学生考虑更灵活的优化方法，并与 `clang -O3` 进行性能比对。机器代码生成同样作为可选实验，供学有余力学生完成，构建更为完整的编译器。

面向于编译、不止于编译：作为实际中软硬件衔接的关键，编译课程在计算机基础教学中也承担着贯穿程序设计、数据结构、组成原理和操作系统等多门课程的作用。因而，实验设计在满足编译教学需要的同时，也应促进学生对基础课程的融会贯通。对此，我们鼓励学生通过对所用精简 C 语言的编译构建，深入理解程序源代码到执行代码的流程，并添加所需语法实现“用自己开发的编译器编译自己的操作系统，并运行在自己设计的处理器上”的目标。此外，借助开源编译库的使用和二次开发，学生能够更好的完成从知识学习到能力提升的转换，提高问题分析求解和构建复杂工程的能力。

4　具体实验方案

围绕上述理念，我们基于 Clang/LLVM 自顶向下地设计了编译器构造实验框架，并使用木兰宽松许可证（第 2 版）[1] 开源于知名代码托管网站 GitHub。自项目开始的半年来已获得上百个星标，

1) https://github.com/arcsysu/SYsU-lang/blob/latest/LICENSE

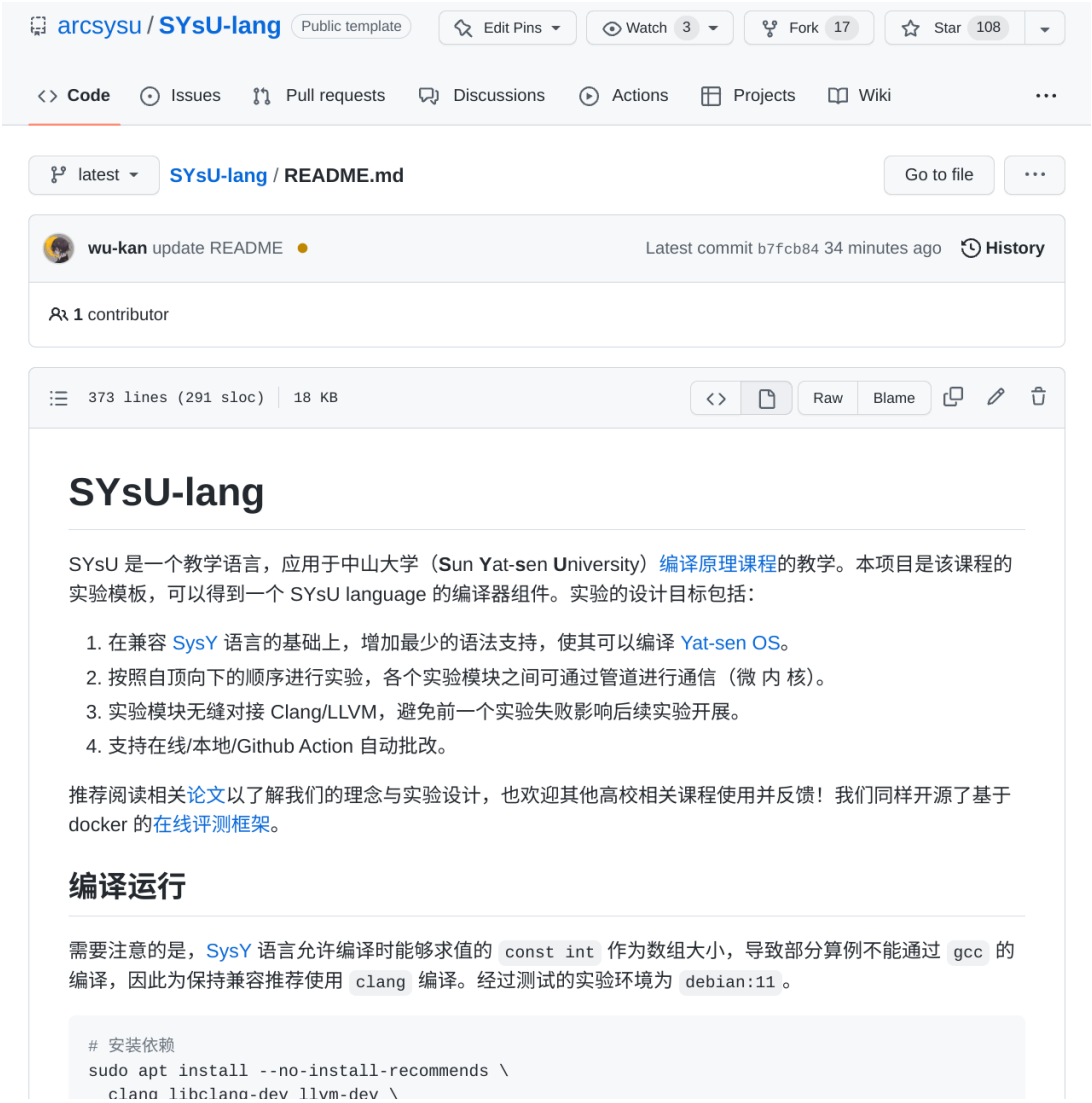

图 2　实验框架开源于 GitHub 平台

见图-2。

与其他高校相比，本实验特色之一是将完整编译器拆分为多个模块（如图-3 所示），其中核心的词法分析、语法分析（语义分析）、中间代码生成、中间代码优化四个过程被设定为基础性必做实验；当学生完成从源代码转换到中间代码的完整流程后，即可认为已经完成一个完整的编译器构建。实验框架中包含了预处理器模块与生成汇编代码的后端模块，但并不强制要求学生必须完成，而是作为挑战项目供有兴趣、学有余力学生完成基本实验外自行实现并替换。除实现编译器的其他模块，完成必做实验本身也可以带来很多有趣的现实问题与挑战，将在下文中予以详细介绍。

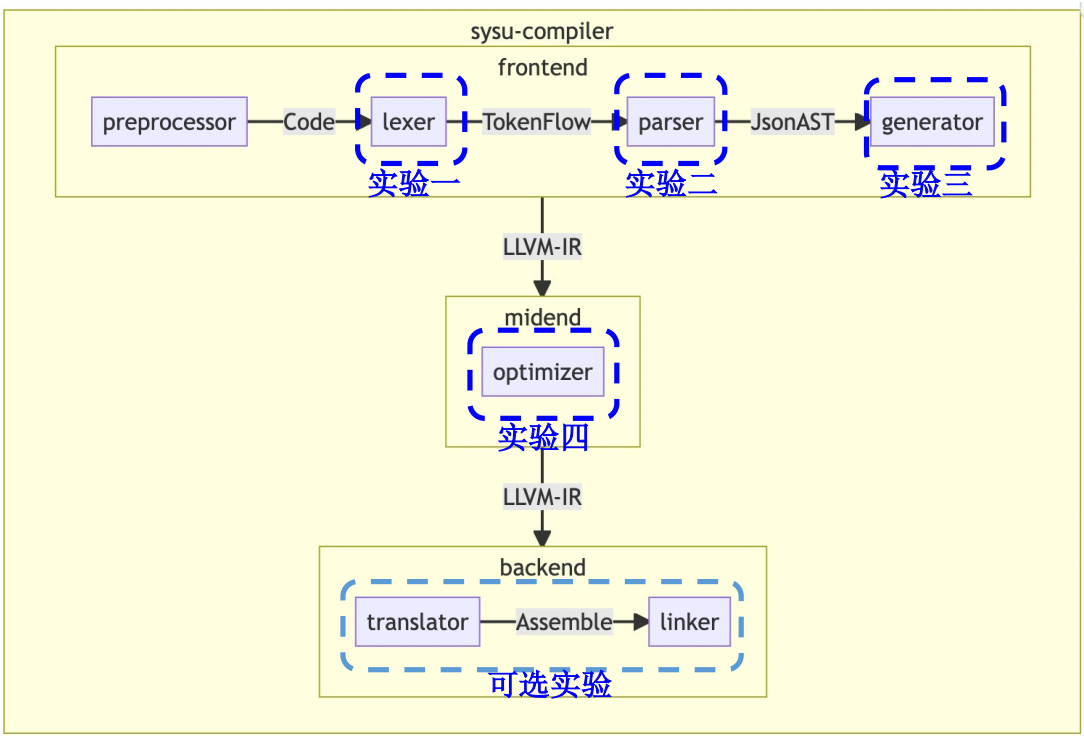

图 3　实验总体安排

4.1　实验一：词法分析器

作为第一个实验，词法分析实验设计上不宜难度过高，否则容易让学生产生畏难、抵触的情绪。作为简化，实验提供了一个约 90 行、基于 `flex` 的模板，学生需要在此基础上扩充其中的词法规则，实现完整的词法分析器，对输入的源代码进行分词、标注词性以及在源代码中的相对位置，完整实现代码量约为 250 行。实验同时提供了基于 `python` 编写的检验脚本，可以将学生的输出与 Clang 标准词法输出（`clang -cc1 -dump-tokens 2>&1`）进行比较，通过约 400 个算例的检查即视为通过。学生可以通过本地执行检验脚本或是 GitHub Action 完成结果检查，耗时约 20 秒～3 分钟，不会影响实验正常进展。

4.2　实验二：语法分析器

自实验二开始，难度阶梯上升，学生将在实践中锻炼自己工程实现能力，并一窥 LLVM 的真面目。实验二工作量上设计为约 1000 行代码。

实验二任务为完成一个语法分析器，接受来自实验一或者 `clang -cc1 -dump-tokens 2>&1` 的输入，输出 JSON 格式的语法树；检测脚本将把 Clang 标准 AST（`clang -cc1 -ast-dump=json`）与学生的输出进行比较，同时遍历两棵语法树，检查每一个节点是否正确。

需要说明的是，考虑到实验二本身的难度跃升，我们对复杂的语法/语义分析进行了简化，并不要求学生手写递归下降法或移进归约来完成这一过程，而是使用成熟的 `bison` 自动解析工具来降低实验难度。基于形式自动机理论实现语法分析的知识已经足够成熟，而掌握 `bison` 工具的使用，对

二、 拓展：sysu-format + sysu-refactor + sysu-tidy(均只有部分及其简单的实现)

1. 原理

将clang-AST作为输入，遍历语法树，从而实现sysu-format + sysu-refactor + sysu-tidy。

由于clang-AST会进行预处理，例如会自动处理引用的头文件并直接将其引入当前文件，从而会影响对于行号的估计以及其他一些影响。如果希望能够完美处理单个文件的静态检查，则需要按照自己的需求编写lexer和parser，由于时间原因，还是选择直接使用clang-AST了。

2. 配置项

通过配置config.json，从而实现不同风格的代码规整化、重构和规范性检查

```
{
    "indent": 4,
    "initialListMode": true,
    "bracePosition": "changeLine",
    "emphasizeType": false,
    "dense": false,
    "emitBrace": false,
    "check":[
        {"-Wdangling-else": true},
        {"-Wunused-value": true},
        {"-Wempty-body": true},
        {"-Wtautological-compare": true}
    ],
    "refactor":[
```

图 4 学生基于实验二实现的小工具

快速实现一门语言的需求更加具有现实意义。另外，实验二的 SYsU 文法是 SysY 文法的一个稍微扩展，文法的 EBNF 定义 [26] 完整，资料丰富，文法本身十分精简，在教师和助教的帮助下，学生预期能够以 1000 行左右的代码量完成实验。

实验二要求输出为 JSON 格式的语法树，并提供了基于 `bison` + `llvm::json` 的约 100 行代码模板。这样的设计至少有以下三点好处：1）便于使用 `python` 脚本和 `clang` 导出的语法树对比，自动批改；2）JSON 格式非常容易理解与上手，不需要像 LLVM 官方教程 [18] 一样定义很多节点；3）可以让学生提前上手 LLVM 开发库的使用，平滑下一个实验的难度。此外，实验模板也尽量避免使用 `bison` 提供的一些机制，而是使用了一个自行构造的栈用于语法分析的中转，让学生理解 “移进-规约” 的过程。例如，遇到悬垂 else 等可能会出现二义性的情况时，`bison` 使用的 LALR(1) 文法会向前多检测一次 token，学生如果不注意便会从栈中取出错误的节点。

经过实验二,学生得到了精简 C 语言的语法树,此时也可以引入关于 `clang-format`、`clang-tidy`、`clang-refactor`、`clangd` 等基于 `clang` 的现代化开发工具介绍。如图-4，学生在体验现代化编译工具带来的便利后，甚至可以额外实现一套简单的代码规范检查、规整化、重构工具。

4.3 实验三：IR 生成器

难度在实验三达到峰值，学生将完全进入 LLVM 开发范畴。本实验要求学生完成一个 IR 生成器，接受来自实验二或者 `clang -cc1 -ast-dump=json` 的输入，输出 LLVM IR；检测脚本会通过 `clang -cc1 -O3 -S -emit-llvm` 得到用于对比的 LLVM IR；两份 IR 将同时通过 `llc -O0` 编译成二进制可执行文件，随后比较该文件执行时的输出与返回值。完成实验三预期需要 1500 行代码。

实验三带给学生的意义，除了完成一个完整的编译器外，还可以自然地启发学生编译器和计算机硬件是怎样认识我们编写的代码的，从而自然地领悟一些优秀的编程习惯。例如 C 语言中的短路运算，看起来有时可以规避掉一些不必要的计算，但是实验中将其转换为中间代码表示时却需要一些复杂的跳转指令，很多时候并不如直接使用位运算。

在学生完成实验三时，一些特殊的算例并不能在评测机的时限内运行完成，此处即可引出实验四的 IR 优化实验。此外还会遇到一个非常有意义的问题：在处理一个语法树深度达到数十万的大算例时，模板中使用的 `llvm::json::parse` 性能难以满足要求，在评测机上需要运行十五分钟才能将其完全解析，而作为对照，`python` 中的 `json.loads` 仅需要八秒。实验中同样允许超时跳过这一算例，但是学生也同样被鼓励使用编译原理课程上学到的知识，实现一个高性能的 JSON 处理库，并与一些高性能的实现做对比。

4.4 实验四：IR 优化器

有别与前三个实验，实验四更具开放性，没有预期完成时间与代码行数。基本要求是，完成一个 IR 优化器，接受来自实验三或 `clang -cc1 -O0 -S -emit-llvm` 的 LLVM IR，输出优化后的 LLVM IR。学生在本实验中需要编写 LLVM Pass，重点面向实验三中的三类超时情况进行优化，即：死代码消除、公共子表达式提取、常量除法转换。

值得一提的是，实验四关注的 Pass/PassManager 是 LLVM 里最重要的核心组件之一，自 LLVM 诞生以来已经有数十年历史。由于原有的 PM 编译效率低且错失很多优化机会，2014 年开始 LLVM 团队对其开始重构。和其他高校的实验相比，我们的实验设计上有后发优势，在实验环境中使用的 LLVM-11 虽然默认使用的仍然是旧 PM，但已经包含新 PM，学生也被鼓励去写新规格的 LLVM Pass。

学生在实验四中的得分由在线评测中与 `clang -cc1 -O3 -S -emit-llvm` 运行时间的比值决定。这样的评分方式下，学生很难获得满分，因此此处鼓励学生完成几个实验中的挑战选项以获取额外的加分。

5 初步评估

基于上述设计，我们在 2022 年春季学期于中山大学计算机学院开展了实验教学。课程为《编译器构造实验》，与《编译原理》同为必修课程，面向计算机科学与技术专业大三学生，跨度为完整学期，共 20 周 1 学分 36 学时。实验课堂规模为 60 人，1 名任课教师，3 名助教。每项实验要求学生提交完成代码和简短实验报告，报告内容，特别是心得部分，不做强制性要求，不计入评分，以获得最真实的反馈。以下为此次教学的初步数据统计和反馈收集。

表 2　实验任务量及时间安排

实验	内容	代码量 (行数/LOC)	用时 (小时/h)	预留时间 (天/d)
一	词法分析器/lexer	250	2 - 6	14
二	语法分析器/parser	1000	24 - 72	30
三	IR 生成器/generator	1500	36 - 108	30
四	IR 优化器/optimizer	开放性实验，推荐值为 500	24 - 150	30

图 5　学生完成实验二之余制作的表情包，苦中作乐

5.1　教学反馈

在实际的教学过程中，我们根据难度和工作量对各实验预留了不同的完成时间（见表-2）。学生完成的时间也由最开始词法分析的 2 天到中间代码生成 3 周的过程，代码量相应由 250 行到 1500 行的递增。

实验一基于 `flex` 识别正则表达式从而完成词法分析大大减低了实验难度，代码量总体约 200 行，耗时约 2 到 6 小时。经学生反馈，其中主要的难点是使用正则表达式描述包含转义字符的 C 语言字符串。此外，超过半数的学生并没有熟练使用命令行与 Unix/Linux 工具链的经验；学生完成实验一期间的提问主要集中在实验环境部署、环境变量、头文件/库文件/可执行文件缺失上，非常适合以此作为切入点，进行一次 Linux 使用与常用指令的教学。

实验二相比实验一，在难度上有明显提升。在实验早期，大部分学生并不完全理解需要做的工作；而少部分能力强的学生也质疑是否能够完成。在课程教学中，我们的解决方法是：由教师制作详细的实验原理指导[2)]，同时建立在线问答平台[3)]，由助教与具有解决经验的学生共同答疑，鼓励学生就遇到的问题多多提问。最终，大三年级学生中约 90% 可以在规定期限内顺利完成，其余学生在适当延期和额外指导下也能陆续提交。如图-5 和图-6 所示，学生在对语法树操作过程中备感挑战的同时也在完成之际收获了极大成就感。

实验三是本编译实验的核心和精髓，同时也是四个实验中挑战最大的一个。中间代码的生成标志着编译器前端工作结束、正式进入中端范畴，生成 LLVM IR 的必备知识包含 LLVM/IR 本身 [19] 的表示规则和生成 LLVM/IR 的 LLVM/IR 库使用方法，二者的理解和掌握需要花费不少时间。实

2) https://xianweiz.github.io/teach/dcs290/slides-22/lab_7.pdf

3) https://github.com/arcsysu/SYsU-lang/discussions

5 实验心得

这次实验和上次相比，难度有很大提升，需要我们对于语法分析的规则，以及移进冲突等有比较清晰的认识和理解。在实验过程中，我也遇到了很多问题。但是好在助教给的 SysY 语言相关材料比较详细，对写文法规则有很大帮助；同时，助教和老师也对实验做了详细的说明，教给我们处理文法冲突的办法，这些都给我们做实验带来很大的启发。总的来说，这次实验花费了不少时间和经历，但是在一步步查看语法树，一步步通过和 clang 标准输出比对进行 debug 的过程中，我对语法分析这个过程也有了更加清晰的认识和理解，在最后通过所有测试例子时，内心充满了成就感。

图 6 学生实验二心得

际编写程序也需要考虑众多问题：比如语法树递归遍历，多维数组处理，if/else 控制分支语句处理，短路运算求值过程等，非常考验学生的算法和数据结构能力。此外，学生在实验三遇到的另一个难题是：LLVM 库本身的更新与重构过于频繁，很多时候文档并没有跟上；优质的中文资料更是偏少。在实验中我们由助教与学生同时完成了实验三，并编写中文指导文档，标注一些实验中可能会使用到的 LLVM API 与使用方法。

实验四是开放性实验，其难度完全由学生自行把控，追求性能优化的同时将伴随编程工作量的上升。但是一些常见的优化技巧，比如死代码消除等资料较为丰富，预计的代码量在 500 行左右。截至论文投稿日（2022 年 5 月 31 日），实验四发布时长不超过一周，但已有学生将编译器的性能从开始的 12%（与 `clang -O0` 相当）提高到 24%，加速比超过 100%，总体来说非常值得期待。

拓展性实验方面，在完成实验基本任务之外，不少学生积极进行探索，尝试完成一些挑战性工作。图-4 展示了学生在实验二中利用 JSON 进行的代码格式规整化和规范检查的工具开发。尽管实验四尚在进行中，部分学生已经对后继机器代码生成进行了思考。

5.2 主要挑战及问题

根据学生的反馈，实验目前的挑战和问题主要体现在以下方面：

1. SYsU 文法和 Clang 语法树：在实验二开展过程中，SYsU 文法实际上是在 SysY 文法基础上稍微进行了扩展，所以文法本身并不会特别复杂。在根据文法实现语法分析的过程，实验中允许学生使用 `bison` 等语法分析工具，没有强制学生手写递归下降或规约推导方法来实现，减少了语法分析的实现复杂性。实验二要求学生生成一棵和 Clang 生成对比的 JSON 语法树，而 Clang 语法树具有明确规律的生成规则和语义，其具体的结点生成规则。如图-6 学生实验报告写道，语法树的规则可以通过自己手动编译测试样例生成 JSON 语法树，进行观察从而掌握。当然，在未来的编译原理实验教学中，我们将给出具体明确的 Clang 的 JSON 语法树生成规则，以进一步降低实验的难度。

2. 教程和帮助文档：由于 LLVM 的文档 [20] 很多时候赶不上其开发速度，一些 API 的功能描述略少，导致学生在着手开展实验时存在短期的不适应。但是，实验中用到的很多 LLVM IR 部

这些变量因为在整个文件的作用域只出现了一种类型的声明，实际上完全可以提出来，在外面声明一次。否则while循环内部一直声明，每次声明都要开辟内存空间，会导致程序的堆栈错误。

为了能够判断是否可以延迟定义变量，我需要遍历两次AST，以及重构符号表，又花了很多精力。

- 第五次重构发生在数组各种奇葩初始化的填坑上

  我完全没有想到为什么会有这么多五花八门的初始化，为了能够应付不同初始化，我又花了很多时间

- 第六次重构发生在类型处理上

  这是我一开始就担心的。我知道我到了最后一个字符画的测例，各种类型的转化、运算、函数调用会让我吃不了兜着走。果然，我遇到了。

此外还有诸多的小细节，每次遇到这些问题，我就想推开凌晨两点的宿舍门从五楼一跃而下。不过好在，我终于写完了。虽然效率不高，llvm的很多API都不会调用，可能有更好的API解决方案，我用自己的笨办法解决了。

这次实验我误到的一点是：作为一个写编译器的，最不该想的就是应付差事。不然凭什么认为自己编译器的代码都可能有问题，编译出来的代码却可以蒙混过关呢？

图 7　学生在实验中会面临需要多次重构的情况

分的 API 并不会特别复杂，完成实验需要使用的 API 数量有限，加上老师和助教的引导，没有成为学生完成实验的绊脚石。然而，如图-7 所示，部分学生还是在实验过程中不得不对代码进行多次重构。为尽量避免此类问题频繁发生，我们后续将对本实验中使用到的 LLVM 相关 API 编写详细实用说明文档，提供给将来开展的编译原理实验教学。与此同时，我们也计划邀请本届学生参与到后续的课程建设和改进中，在代码和文档方面进行持续性改进。

这些挑战于我们是实验的改进方向，于学生是潜在的经验财富。通过实验历练，学生对 CMake、Linux 命令等系统开发工具和技术的掌握，无论将来是否从事编译领域的研究或工作，都将裨益其计算机职业生涯。

6　总结与展望

为更好的满足现代计算机专业领域所要求的人才培养需求，在综合分析和借鉴国内外相关课程设置的基础上，我们面向中山大学计算机专业教学实际，设计了基于 Clang/LLVM 的模块化和解耦化编译实验，在聚焦编译核心技术学习和掌握的同时，侧重培养学生的系统观、全局观。初步的教学结果表明，所设计的编译教学实验有效提高了实践性和实用性。

后续我们将对实验进行持续性完善和提升，一些可能的后续计划如下：参照 `flang` 等开源编译器实现，探索将 MLIR 等 LLVM 前沿组件引入课程实验的可能性；完善可选的后端（RISC-V, ARM, X86）与 JIT 模块实验设计，供有兴趣、有能力的学生挑战；使用 SYsU 文法实现各个实验模块，实现编译器的自举；将 SYsU-lang 加入教育平台超算习堂的在线实训[4)]，进一步降低实验门槛、扩大受众范围；与中山大学操作系统、组成原理实验课程进行软硬协同设计，使用相同的实验环境，以最少

4) https://easyhpc.net/lab

的代码量达成“用自己开发的编译器[5]编译自己的操作系统[6]，并运行在自己设计的处理器[7]上”这一目标，培养出时代真正需要的、具有软硬系统大局观的学生。

参考文献

1 高教创时代，教育部全面实施教育数字化战略行动构建中国特色高等教育数字化新体系，https://m.eol.cn/toutiao/202203/t20220328_2217408.shtml

2 教育部，教育部高等教育司 2022 年工作要点, http://www.moe.gov.cn/s78/A08/tongzhi/202203/W020220310547779354544.pdf

3 Colfaxresearch, A Performance-Based Comparison of C/C++ Compilers, https://colfaxresearch.com/compiler-comparison/

4 Chris Lattner, ASPLOS Keynote: The Golden Age of Compiler Design in an Era of HW/SW Co-design, https://tenstorrent.com/research/asplos-keynote-the-golden-age-of-compiler-design-in-an-era-of-hw-sw-co-design-by-dr-chris-lattner/

5 NVIDIA, CUDA LLVM Compiler, https://developer.nvidia.com/cuda-llvm-compiler

6 Philip C. Roth, Experiences with the Heterogeneouscompute Interface for Portability (HIP) on OLCF Summit, https://www.olcf.ornl.gov/wp-content/uploads/2019/10/Roth-HIP-on-Summit-20191009.pdf

7 Coral, Edge TPU Compiler, https://coral.ai/docs/edgetpu/compiler

8 Intel, Intel C/C++ compilers complete adoption of LLVM, https://www.intel.com/content/www/us/en/developer/articles/technical/adoption-of-llvm-complete-icx.html

9 IBM, IBM C/C++ and Fortran compilers to adopt LLVM open source infrastructure, https://developer.ibm.com/blogs/c-and-fortran-adopt-llvm-open-source

10 LLVM, MLIR: Multi-Level IR Compiler Framework, https://mlir.llvm.org/

11 Apache, Apache TVM: An End to End Machine Learning Compiler Framework for CPUs, GPUs and accelerators, https://tvm.apache.org/

12 Robin Zhong, LLVM x Blockchain: A New Ecosystem of Decentralized Applications, https://llvm.org/devmtg/2018-04/slides/Zhong-LLVM%20x%20Blockchain.pdf

13 Sukrit Kalra and Seep Goel, et al. ZEUS: Analyzing Safety of Smart Contracts, In Network and Distributed Systems Security (NDSS) Symposium, San Diego, CA, USA, 2018.

14 Microsoft, Introducing Quantum Intermediate Representation (QIR), https://devblogs.microsoft.com/qsharp/introducing-quantum-intermediate-representation-qir/

15 Pradnya Khalate and Xin-Chuan Wu, et al. An LLVM-based C++ Compiler Toolchain for Variational Hybrid Quantum-Classical Algorithms and Quantum Accelerators, https://arxiv.org/pdf/2202.11142.pdf

16 华为, 毕昇编译器介绍, https://support.huaweicloud.com/ug-bisheng-kunpengdevps/kunpengbisheng_06_0001.html

17 百度, 源于产业实践的开源深度学习平台，https://www.paddlepaddle.org.cn/

18 LLVM Compile Infrastructure, LLVM Language Reference Manual, https://releases.llvm.org/11.0.1/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

19 LLVM Tutorial, Kaleidoscope: Implementing a Parser and AST, https://releases.llvm.org/11.0.1/docs/LangRef.html

20 LLVM Compile Infrastructure, Using the New Pass Manager, https://llvm.org/docs/NewPassManager.html

21 Stanford University, CS143: Compilers, https://web.stanford.edu/class/cs143/

22 Cornell University, CS 6120: Advanced Compilers, https://www.cs.cornell.edu/courses/cs6120/2019fa/project/

23 清华大学, MiniDecaf 编译实验, https://github.com/decaf-lang/minidecaf-tutorial

24 北京大学, PKU compiler course online documentation, https://github.com/pku-minic/online-doc

25 北京航空航天大学, miniSysY 编译实验, https://github.com/buaa-se-compiling/miniSysY-tutorial

26 全国大学生计算机系统能力大赛——编译系统设计赛，SysY 语言定义，https://gitlab.eduxiji.net/nscscc/compiler2021/-/blob/master/SysY%E8%AF%AD%E8%A8%80%E5%AE%9A%E4%B9%89.pdf

5) https://github.com/arcsysu/SYsU-lang

6) https://github.com/YatSenOS/YatSenOS-Tutorial-Volume-1

7) https://yatcpu.sysu.tech